

Mlvan Rojas's Org
Scan Report



Application: descargatue4
Environment: Pre-Production
Completed On: Wed, 30 Jul 2025 19:55:49 GMT

Application: descargatue4
Environment: Pre-Production

Scan Date: Wed, 30 Jul 2025 19:55:49 GMT | Paths Scanned: 12 | Scan Duration: 515s

High	Medium	Low	New	Assigned	Risk Accepted	False Positive
0	2	4	6	0	0	0

Criticality	CWE	Finding	New	Assigned	Risk Accepted	False Positive
Medium	CWE-693	Content Security Policy (CSP) Header Not Set	2	0	0	0
Low	CWE-693	Permissions Policy Header Not Set	3	0	0	0
Low	CWE-1004	Cookie No HttpOnly Flag	1	0	0	0

Content Security Policy (CSP) Header Not Set

Criticality: Medium CWE ID: 693

Description:

Remediation

To fix the "Content Security Policy (CSP) Header Not Set" vulnerability, you need to properly configure the Content Security Policy header in your web server or application. Here are the steps to remediate this issue:

1. Identify the web server or application: Determine the technology stack used for your website or application. This could be Apache, Nginx, IIS, or a specific web framework like Express.js or Django.

2. Configure the Content Security Policy header: Add the Content Security Policy header to your server or application configuration. The exact method will depend on the technology stack you are using.

- For Apache, add the following line to your .htaccess file or Apache configuration file:

```
Header set Content-Security-Policy "default-src 'self';"
```

- For Nginx, add the following line to your server block or location block:

```
add_header Content-Security-Policy "default-src 'self';";
```

- For IIS, open the IIS Manager, select your website, and go to the "HTTP Response Headers" section. Add a new header with the name "Content-Security-Policy" and the value "default-src 'self';".

- For specific web frameworks, consult the documentation for that framework on how to set the Content Security Policy header.

3. Customize the Content Security Policy: The example above sets a basic Content Security Policy that only allows resources to be loaded from the same origin. However, you may need to customize the policy based on your website's requirements. Refer to the Content Security Policy specification (<https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>) for more information on the available directives and their usage.

4. Test and monitor: After configuring the Content Security Policy header, thoroughly test your website or application to ensure that all resources are being loaded correctly. Monitor your application's security logs and reports to detect any potential violations or issues with the Content Security Policy.

About

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross-Site Scripting (XSS) and data injection attacks. These attacks can be used for various malicious purposes, such as data theft, site defacement, or distribution of malware.

CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on a page. The covered types of content include JavaScript, CSS, HTML frames, fonts, images, and embeddable objects like Java applets, ActiveX, audio, and video files.

By properly configuring the Content Security Policy header, you can restrict the loading of content from unauthorized sources and reduce the risk of XSS attacks and other malicious activities.

Risks

Failure to set the Content Security Policy (CSP) header exposes your website or application to various security risks:

- 1. Cross-Site Scripting (XSS) attacks:** Without a proper CSP, attackers can inject malicious scripts into your web pages, leading to XSS attacks. These attacks can result in the theft of sensitive user information, session hijacking, or defacement of your website.
- 2. Data injection attacks:** Attackers can inject malicious data into your website or application, leading to data corruption, unauthorized access, or the execution of arbitrary code.
- 3. Malware distribution:** Without a CSP, attackers can load and execute malicious files on your website, leading to the distribution of malware to your users.
- 4. Site defacement:** Attackers can modify the appearance or content of your website, damaging your brand reputation and causing user distrust.

By implementing a proper Content Security Policy, you can significantly reduce the risk of these security threats and protect your website or application from potential attacks.

Path	Method	Status
[ROOT]	GET	NEW
/build/assets/app-D80eutCD.js	GET	NEW
/sitemap.xml	GET	NEW

Permissions Policy Header Not Set

Criticality: Low CWE ID: 693

Description:

Remediation

To fix the "Permissions Policy Header Not Set" vulnerability, you need to add the Permissions-Policy header to your web server configuration. This header allows you to specify which browser features can be used by your web resources. By setting appropriate permissions, you can restrict unauthorized access or usage of sensitive features such as camera, microphone, location, and full screen.

To add the Permissions-Policy header, follow these steps:

1. Apache: Add the following line to your .htaccess file or Apache configuration file:

```
Header set Permissions-Policy "camera=(), microphone=(),  
geolocation=(), fullscreen=()"
```

2. Nginx: Add the following line to your Nginx configuration file within the http block:

```
add_header Permissions-Policy "camera=(), microphone=(),  
geolocation=(), fullscreen=()";
```

3. IIS: Open the IIS Manager, select your website, and go to the "HTTP Response Headers" feature. Add a new header with the name "Permissions-Policy" and the value "camera=(), microphone=(), geolocation=(), fullscreen=()".

Make sure to adjust the permissions according to your specific requirements. For example, if your website requires camera access, you can remove the camera=() part from the header value.

About

The Permissions Policy Header is an added layer of security that helps restrict unauthorized access or usage of browser/client features by web resources. It ensures user privacy by limiting or specifying which features of the browser can be used by web resources. By setting the Permissions-Policy header, website owners can control access to features such as camera, microphone, location, and full screen.

Risks

If the Permissions Policy Header is not set, it can lead to potential security risks. Without proper restrictions, malicious actors may exploit browser features to gain unauthorized access to sensitive user information or perform unauthorized actions. For example, an attacker could abuse

camera or microphone access to capture audio/video without user consent, or misuse geolocation data for tracking purposes. By not setting the Permissions Policy Header, you leave your website and users vulnerable to these risks.

Path	Method	Status
[ROOT]	GET	NEW
/build/assets/app-D80eutCD.js	GET	NEW
/sitemap.xml	GET	NEW

Cookie No HttpOnly Flag

Criticality: Low CWE ID: 1004

Description:

Remediation

To remediate the "Cookie No HttpOnly Flag" vulnerability, the following steps can be taken:

- 1. Enable the HttpOnly flag:** Set the HttpOnly flag on all cookies to prevent them from being accessed by JavaScript. This can be done by adding the HttpOnly attribute to the Set-Cookie header when the cookie is being set. For example, in a web application using PHP, the `setcookie` function can be used with the `httponly` parameter set to `true`:

```
setcookie('cookie_name', 'cookie_value', time() + 3600, '/', '',  
false, true);
```

This will ensure that the cookie is only accessible over HTTP and cannot be accessed by JavaScript.

- 2. Review and update cookie handling:** Review the codebase to ensure that cookies are being handled securely. Make sure that sensitive information is not being stored in cookies and that cookies are properly validated and sanitized before being used.
- 3. Implement secure session management:** If the vulnerable cookie is a session cookie, consider implementing additional security measures such as session expiration, session regeneration after login, and using secure session storage mechanisms.

About

The "Cookie No HttpOnly Flag" vulnerability occurs when a cookie is set without the HttpOnly flag. The HttpOnly flag is a security feature that prevents cookies from being accessed by JavaScript. By default, cookies can be accessed by both the server and client-side scripts, including malicious scripts. If a malicious script is able to run on the same page as the vulnerable cookie, it can access and potentially steal the cookie's value. This can lead to session hijacking, where an attacker gains unauthorized access to a user's session.

Risks

The risks associated with the "Cookie No HttpOnly Flag" vulnerability include:

- 1. Session hijacking:** If the vulnerable cookie is a session cookie, an attacker can potentially hijack a user's session by stealing the cookie's value. This can allow the attacker to impersonate the user and perform actions on their behalf.

- 2. Information disclosure:** If the cookie contains sensitive information, such as user credentials or personal data, an attacker can access and steal this information. This can lead to identity theft, unauthorized access to accounts, or other privacy breaches.
- 3. Cross-site scripting (XSS) attacks:** The ability to access cookies through JavaScript can be exploited in conjunction with other vulnerabilities, such as cross-site scripting (XSS). An attacker can inject malicious scripts into a vulnerable page, which can then access and steal the cookie's value. This can lead to further attacks, such as session hijacking or data manipulation.

Path	Method	Status
[ROOT]	GET	NEW